>> Intelligent Systems – WC 2 <<

Informed and Adversarial Search

General information WC

- Post a message on Slack if you have questions: we will try to answer within 24 hours on workdays.

- In WC you will practice with exercises to understand the theory of this course
- Preparation for the exam
- !!Make sure you have paper and pen with you!!
- Prepare yourself for WC
  - Look up the slides
  - Watch videos
  - Read relevant material in the book
  - Try to make the WC exercises beforehand

Concepts to be known after WC2 (Uninformed search + Informed search)

Representation of state-space problems:

Uninformed search:

Informed search

- Hill-climbing (= local greedy search)
- Best-first search (= global greedy search)
- A
- A*
- Admissible

Adversarial search/Games

- MiniMax
- Alpha/beta pruning

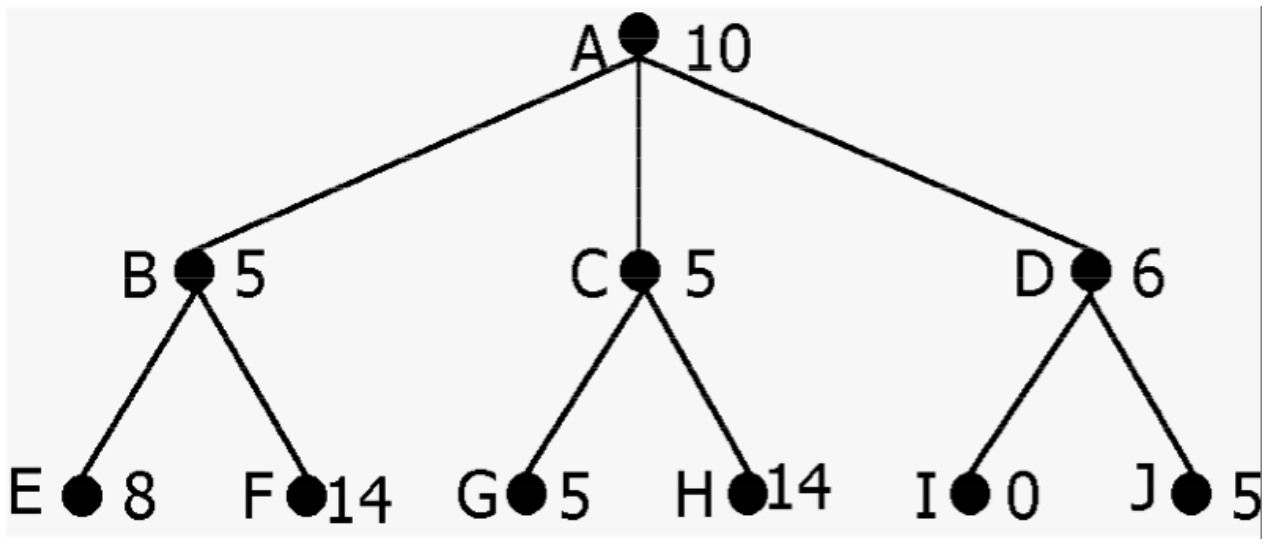
Reminder: Tree Search algorithm (pseudocode)

```
function TREE-SEARCH(problem,fringe) return a solution or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if EMPTY?(fringe) then return failure
        node ← REMOVE-FIRST(fringe)
        if GOAL-TEST[problem] applied to STATE[node] succeeds
            then return SOLUTION(node)
        fringe ← INSERT-ALL(EXPAND(node, problem), fringe)
```

Exercise 1: When do you advise to search data-driven (or 'data-directed' or 'forward searching') instead of goal-driven (or 'goal-directed' or 'backward searching')?

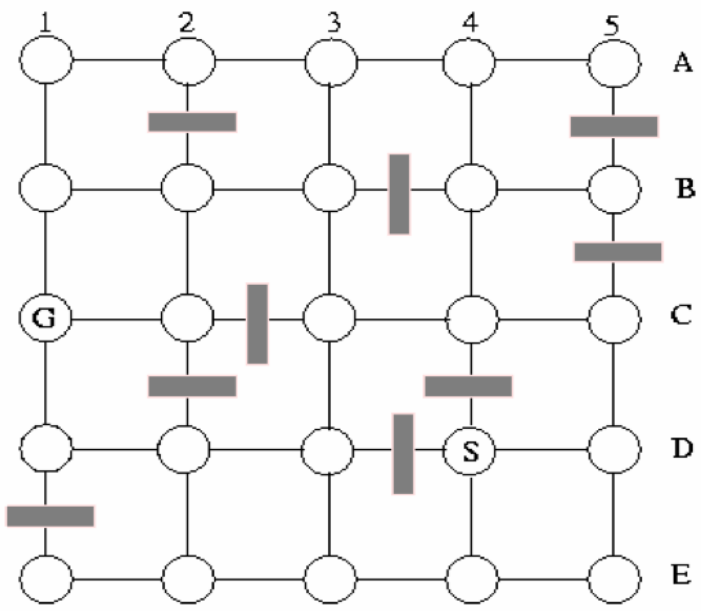
Exercises informed search

Exercise 2: Consider the following state-space, in which node I is a goal node. Furthermore, the lower the heuristic values, the better.



a) What is the order in which the nodes are searched by using **hill climbing**? Write down the letters of the nodes. Choose in alphabetical order when necessary.
b) What is the order in which the nodes are searched by using **greedy best-first search**?

Exercise 3: Given the following maze:



Goal of the maze is to search a route from start position D4 (marked 'S') to goal position C1 (marked 'G').

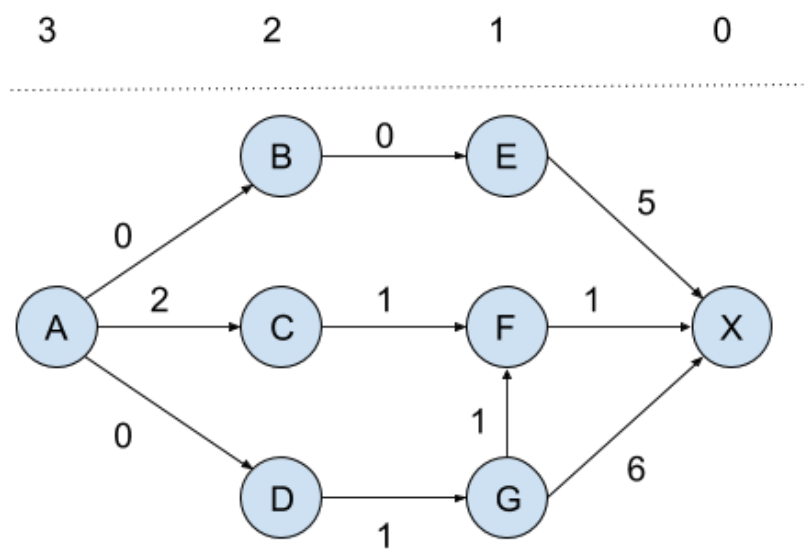
Draw for this maze the search tree by applying a data-driven A* algorithm that uses the Manhattan distance as heuristic h(n). This means that h(n) is just an addition of the horizontal and vertical distances, without taking into account the walls. Which path will eventually be searched by A*? Number the nodes in visiting order by the algorithm. Previously searched nodes with equal or higher values can be ignored. Furthermore, write down the used heuristic value of each node. Use alphabetical order when you encounter equal values.

Exercise 4: Fill in the gaps with ‘greedy, ‘admissible’, ‘underestimates’, ‘overestimates, ‘node’, ‘goal’. (not necessarily all).
A heuristic is _____ if it never _____ the costs to reach a _____.

Exercise 5: Which of the following search methods is/are *optimal*?

a) Breadth-first
b) Depth-first
c) Depth-first iterative deepening
d) Hill-climbing
e) Best-first
f) A
g) A*
h) Beam

Exercise 6: Consider the following search problem:

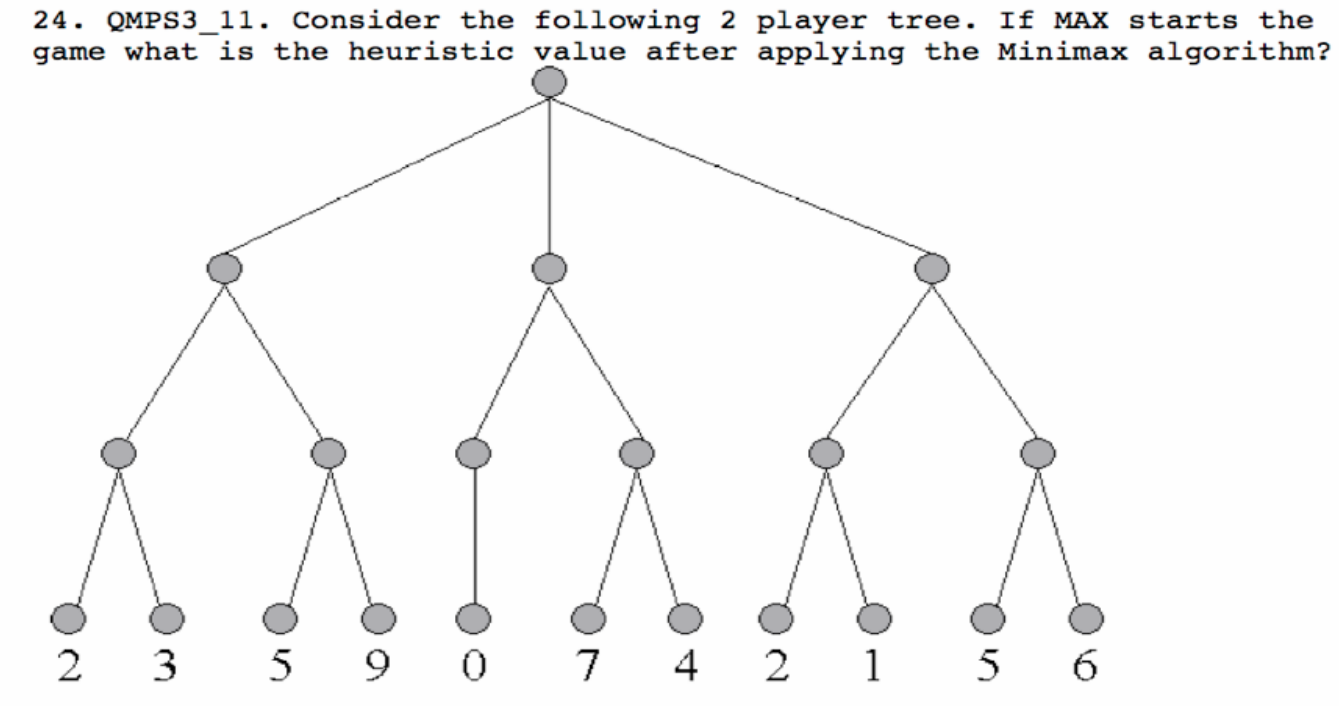


We start in A and want to reach X. The edge labels are the cost of moving from one node to another, and we want to find the path from A to X with the smallest total cost. Draw the A* search tree for this problem. The numbers above the dotted line represent the estimated cost to the goal state for all nodes below it (i.e. for nodes E, F and G, the estimated cost to the goal is 1, and the actual costs are 5, 1 and 6 respectively).
When two nodes have equal value, choose the one with the lowest letter alphabetically to expand first.
We say that a node is *added* when its evaluation function is computed. According to the algorithm from the lecture, only when a node is expanded it is checked whether it is a solution or not. A node is *expanded* when its children are added.

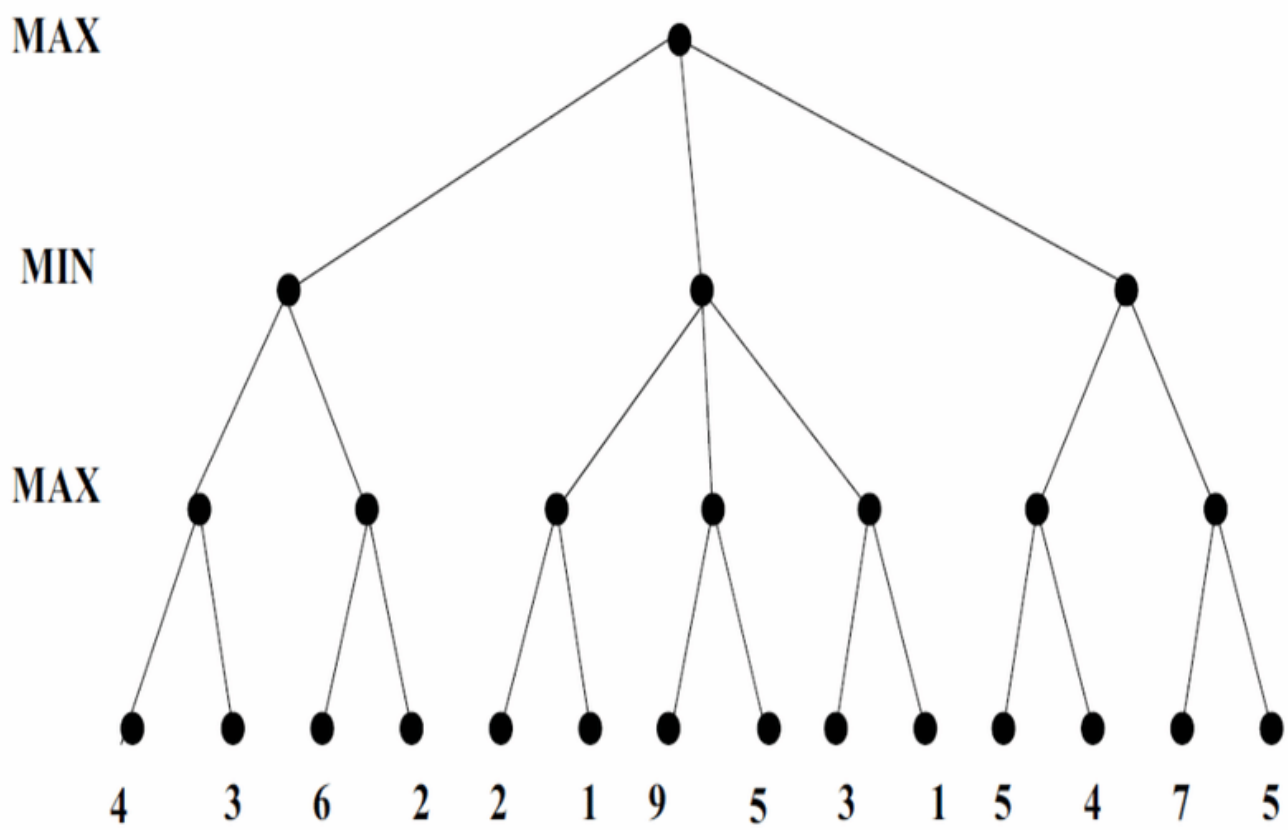
- Do the estimates above the dotted line lead to an admissable evaluation function (answer yes/no
- How often is the goal state added?
- Which node is never expanded?
-

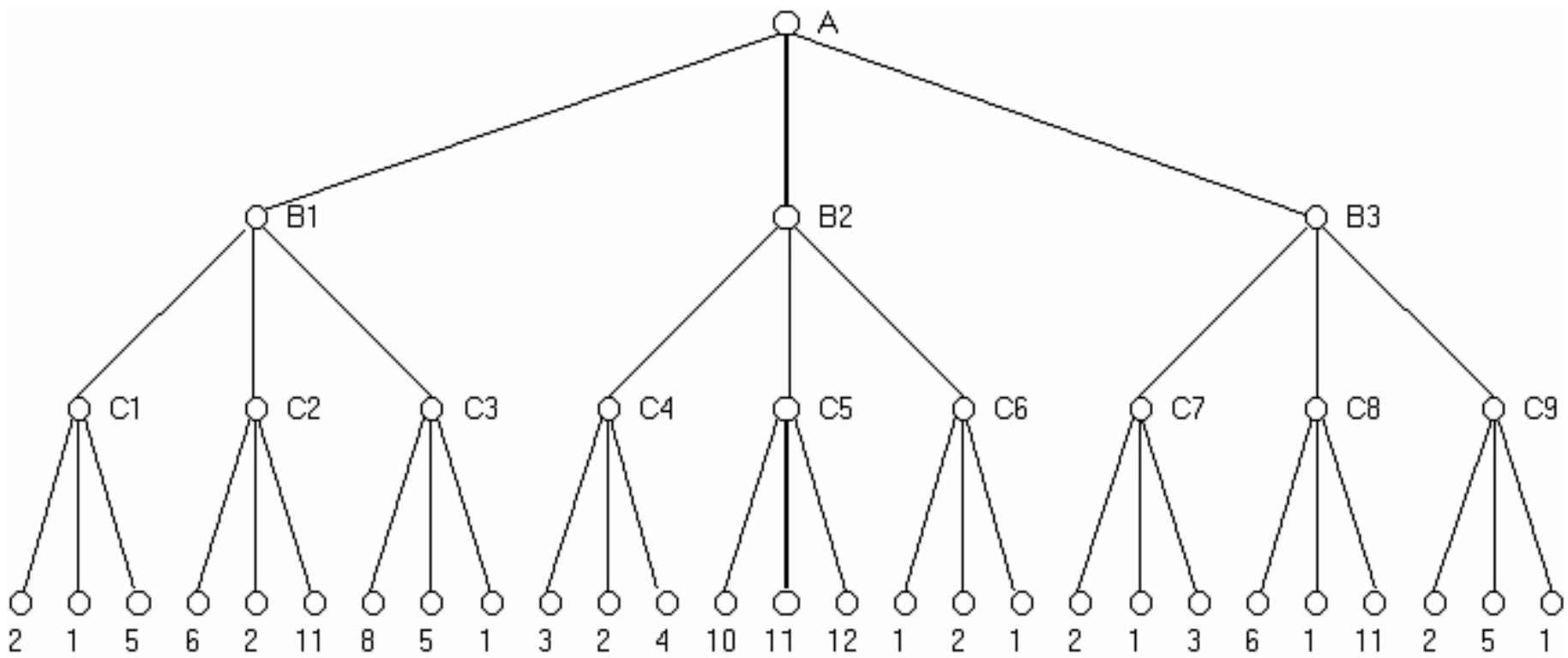
Exercises adversarial search/games

Exercise 7: Consider the following 2 player game tree. If MAX starts the game, what is the heuristic value after applying the Minimax algorithm?



Exercise 8: Consider the following search tree that contains heuristic values at the leaves. The player on the move is depicted at each level. Which nodes can be ignored if we apply alpha-bèta pruning if the goal for Min is to have the smallest possible value, and for Max to have the largest possible value.



Exercise 9: True or false? If MAX plays optimally using the MinMax algorithm, and is allowed to start, it will always find a winning strategy.

Exercise 10: Consider the following MinMax tree. Use the MinMax algorithm to compute what the heuristic value is at position A for MAX when both players play optimally. Assume that MAX makes the first move.